Patchline: Deterministic, Provenance-Carrying Analysis of Data-Change Risk Across the Polyglot Software Stack

The Patchline Project

June 2, 2026

Abstract

Schema migrations, NoSQL change scripts, data-pipeline jobs, and serialization-schema edits are among the highest-risk changes a team ships: they can silently destroy data, break running consumers, or race a deploy, and they are poorly served by the linters and code-scanners built for ordinary source code. We present Patchline, a *deterministic, non-AI-first* analyzer that treats the data-change surface of a real repository as a first-class object. Patchline fetches a project reproducibly from any of several source hosts, inventories what is actually present, derives a project-native *fact layer* with full provenance, ranks data-change risks, and—only when asked, and only within an explicit budget—generates bounded interventions that are quarantined and then *deterministically re-analyzed and rejected* if they do not improve the evidence. Critically, every capability in Patchline is backed by a reproducible *gate*: a script that proves the capability against real, pinned public code and fails loudly on drift. Patchline currently ships 159 such gates over 116 documented capabilities, spanning relational migrations across nine ecosystems, five NoSQL engines, four data-pipeline frameworks, protobuf/Avro schema compatibility, infrastructure/data ordering, and a research-grade evaluation methodology that includes blinded false-positive adjudication, seeded false-negative recall, ablations, standardized effect sizes, and severity calibration against independent danger evidence. We describe Patchline's design principles, architecture, breadth of analysis, intervention-safety model, runtime/observability evidence integration, and reproducibility methodology, and we report results on real repositories including `lobsters/lobsters`, `apache/avro`, `helm/charts`, and a multi-framework lakehouse.

Contents

1 Introduction

A surprising fraction of serious production incidents originate not in application logic but in *data changes*: a migration that drops a column still read by an old replica, a NoSQL cleanup script that truncates the wrong keyspace, a Spark job that overwrites a warehouse table, an Avro field added without a default that breaks every stored record, or a migration Job that runs before the deploy

that depends on it. These changes share three properties that make them dangerous and hard to review:

1. **They are destructive and often irreversible.** Unlike a logic bug, a dropped table or an overwritten partition cannot be rolled back by reverting a commit.
2. **They are polyglot and cross-cutting.** The relevant evidence is spread across SQL, ORM models, framework migration runners, Kubernetes and Helm manifests, Terraform, pipeline code, serialization schemas, CI configuration, and operational notes.
3. **They are poorly served by existing tooling.** Code scanners look for injection and secrets; SQL linters check a single statement's style; AI assistants will happily generate a migration but cannot *prove* it is safe.

PATCHLINE is built on a deliberately contrarian thesis: for data-change risk, a *deterministic* analyzer that carries provenance and refuses to over-claim is more useful, more reviewable, and more publishable than an AI-first generator. PATCHLINE does not require labeled data, a bespoke schema format, or a model API key. You point it at the files a team already has—a GitHub (or GitLab, Bitbucket, SourceHut, Mercurial, or Fossil) repository, a migration directory, a service source tree, a telemetry export, or an incident note—and it inventories what is there, links the clues, ranks the risks, and prints the next commands that can run immediately.

Contributions. This paper makes the following contributions:

1. A **deterministic, provenance-carrying fact layer** for data-change risk that is derived from real repositories without pre-authored schemas (Section 4).
2. A **breadth of analysis** that unifies relational migrations (nine ecosystems), NoSQL changes (five engines), data pipelines (four frameworks), protobuf/Avro schema compatibility, and infrastructure/data ordering under one fact model (Section 5).
3. A **bounded-intervention-with-deterministic-rejection** model that quarantines generated code and re-analyzes it, proving that plausible-but-unsafe changes are rejected before they can be trusted (Section 6).
4. A **runtime and observability evidence** layer that scores findings on independent static-risk and observed-runtime axes using OpenTelemetry, Datadog-style, and Prometheus/Grafana exports (Section 7).
5. A **gate-backed reproducibility and evaluation methodology**—159 gates, blinded false-positive adjudication, seeded false-negative recall, ablations, standardized effect sizes, and severity calibration—designed for artifact evaluation and best-paper-grade rigor (Sections 8–9).

A motivating scenario. Consider a team shipping a routine-looking pull request: a Rails migration that removes a deprecated column, a companion dbt model switched to `materialized='table'`, and a Helm chart that adds a migration `Job`. Reviewed in isolation, each diff looks benign. Reviewed together, they encode a latent incident: the column is still read by a canary replica, the dbt change silently drops and recreates a warehouse table, and the migration `Job` carries no Helm hook, so it may run *after* the deploy that assumes the new shape. No single existing tool sees all three. A code scanner ignores all of them; a SQL linter sees only the `ALTER TABLE`; an AI assistant

might rewrite the migration but cannot prove the ordering is safe. PATCHLINE ingests the whole slice, emits a uniform fact for each hazard with provenance, ranks them by blast radius, classifies the `Job` as *unordered*, and prints the exact follow-up commands— without executing anything and without claiming the migration is "fixed." This scenario recurs, in different ecosystems, throughout the evaluation.

Why determinism, now. The prevailing direction in program-repair research is to let a large model propose and the test suite filter. That loop is powerful for logic bugs with strong oracles, but it is a poor fit for destructive data changes, where the oracle is weak (you cannot "test" a `DROP` in production), the cost of a wrong answer is catastrophic and irreversible, and reviewers must *audit* the reasoning rather than trust a confidence score. PATCHLINE therefore inverts the loop: a deterministic analyzer produces auditable evidence first, and generation is a bounded, quarantined, re-checked afterthought. The remainder of this paper argues—and gate-proves against real public code—that this inversion is both more useful in practice and more rigorous as a research artifact.

2 Design Principles

PATCHLINE's design is governed by five principles that recur throughout the system.

P1: Determinism over cleverness. Every analysis is a pure function of pinned inputs. Given the same archive hash, parser version, and configuration, PATCHLINE produces byte-identical outputs. This makes results cacheable, diff-able across releases, and trustworthy as evidence.

P2: Provenance is mandatory. Every fact records where it came from: the source host, the resolved commit, the file, and the textual signal that produced it. A finding without provenance is not emitted.

P3: Conservative by construction. PATCHLINE prefers a false negative to a confident false positive. Detectors are *signal-gated*: a NoSQL rule only fires when an engine-specific signal confirms the file targets that engine, so prose mentioning "drop the plan" is never misclassified as a `DROP`.

P4: Never over-claim. PATCHLINE does not claim to "repair" a database. It surfaces risk, generates *reviewable* guards and tests, quarantines them, and re-checks them. Generated code is non-executable and unapplied unless a human explicitly opts in to allowlisted safe checks.

P5: Everything is gated. A capability that is not proven against real, pinned public code by a reproducible gate does not count. Documentation, claims, figures, and release notes are all checked against generated evidence; drift fails the build.

3 System Architecture

PATCHLINE is a single Go binary organized as a pipeline of stages, each of which writes deterministic artifacts consumed by the next stage and by the gate layer.

Stage	Responsibility	Key artifacts
`fetch`	Reproducibly resolve and download a source slice from GitHub, GitLab, Bitbucket, SourceHut, a local path, an archive URL, or a Mercurial/Fossil working tree; record provenance and a content-addressed archive hash.	`source.json`
`inventory`	Scan the slice; detect languages, frameworks, migration systems, package boundaries; emit the fact layer.	`inventory.json`, `facts.jsonl`, `project-map.md`
`intake`	Normalize heterogeneous inputs (logs, telemetry, incident notes) into the same fact vocabulary.	`intake` artifacts
`baseline`	Rank data-change risks; estimate blast radius; minimize proof holes; link trace-to-code evidence.	`baseline.json`, `baseline.sarif`
`propose`	Generate bounded interventions (guards, tests) within an explicit budget; quarantine them as untrusted artifacts.	`proposal.patch`, `patchline-proposals/`
`compare`	Deterministically re-analyze proposed changes; accept or reject by evidence delta.	`compare.json`
`deep`	Optional deeper semantic passes (dataflow, lock duration, tenant boundaries).	deep analysis reports

A cross-cutting *gate layer* sits beside the pipeline. Each gate is a self-contained script that (a) validates a versioned example specification, (b) checks that the relevant documentation and the README still describe the capability, (c) runs the capability against real pinned code, (d) asserts machine-checkable invariants on the output, and (e) writes a `gate-summary.json`. The repository currently ships **159 gates**, **116 capability documents**, and **92 example specifications**, with a single analyzer core of roughly 2,900 lines of Go.

Resumability and offline operation. `repo analyze -resume` reuses existing fetch, inventory, intake, baseline, proposal, and compare artifacts while changing later settings, and `repo offline` validates cached inputs and generated reports without any network access—a prerequisite for restricted artifact-evaluation environments.

3.1 Artifact contract

Every stage writes a stable set of files so that downstream stages, gates, and reviewers can rely on their presence and shape. Table 2 summarizes the contract; the determinism principle (P1) means each file is byte-stable for fixed inputs, which is what allows the *redaction-stability* and *reproducibility-report* gates to assert byte-equality across repeated and resumed runs.

Artifact	Contents
`source.json`	Source host, VCS kind, resolved commit, archive SHA-256, fetch timestamp, tool version, cache metadata.
`inventory.json`	Languages, frameworks, migration systems and roots, package boundaries, and all derived data-change surfaces.

Artifact	Contents
`facts.jsonl`	One provenance-carrying fact per line; the canonical machine-readable evidence stream.
`project-map.md`	Human-readable map of the scanned slice.
`baseline.json` / `.sarif`	Ranked risks, invariant candidates, privacy/lock hazards, blast-radius estimates, proof-hole minimizations, and trace-to-code links; SARIF for IDE/CI ingest.
`prompt-context.json`, `prompt.txt`	The exact, minimized evidence sent to a generator, plus excluded-context counts.
`proposal.patch`, `patchline-proposals/`	Generated, quarantined, non-executable artifacts.
`compare.json`	Per-intervention accept/reject decision with the evidence delta that justified it.
`commands.md`	A copy/paste maintainer report with one-command and staged equivalents.

Table 2: The deterministic artifact contract. Each file is byte-stable for fixed inputs.

4 The Project-Native Fact Layer

The core abstraction in PATCHLINE is the *fact*: a small, provenance-carrying record with a kind, a path, a confidence, a human-readable rationale, a set of typed identifiers, and a property map. Facts are emitted as newline-delimited JSON (`facts.jsonl`) so they can be grepped, diffed, and loaded into downstream tools without a database.

```
{"kind":"nosql_change","path":"schema/001_drop.cql","confidence":"observed",
"identifiers":[{"kind":"nosql_engine","value":"cassandra"}],
"properties":{"engine":"cassandra","operation":"dropTable","destructive":"true"}}
```

Three properties of the fact layer matter for the rest of the paper:

- **No pre-authored schema.** PATCHLINE infers schema evolution directly from migration and source text; it does not require the user to first declare a PATCHLINE-specific schema. A bare `DROP TABLE legacy_sessions;` yields a `schema_evolution` fact with `operation=drop_table` and the affected table as a typed identifier.
- **Uniform vocabulary across surfaces.** Relational, NoSQL, pipeline, schema, and infrastructure detectors all emit facts in the same shape, so a reviewer (or a ranking model, or a paper table) can reason over them uniformly.
- **Searchable destructiveness.** Destructive operations carry an explicit `destructive=true` (or `breaking=true`) property, making the highest-risk subset a single grep away.

Sources and reproducibility. The `fetch` stage records the source host, the kind of version-control system (including Mercurial and Fossil working trees, which are content-addressed when no native binary is available), the resolved commit, and a SHA-256 archive hash. This is what makes P1 (determinism) and P2 (provenance) concrete: a baseline computed today can be re-derived byte-for- byte from the recorded provenance.

5 Breadth of Data-Change Analysis

PATCHLINE's distinguishing feature is that it treats *every* place data changes destructively as a first-class surface, not just relational migrations. Each surface below is signal-gated (P3), emits uniform facts, and is proven against real public code by a dedicated gate.

5.1 Relational migrations across nine ecosystems

PATCHLINE detects migration systems and their *native commands* for Rails, Django, Alembic, Prisma, TypeORM, Liquibase, Flyway, EF Core, and Go migrators, and additionally Laravel, Ecto, Diesel, Sequelize, Knex, Doctrine, and Rails multi-database setups. Crucially, each detection maps to the project-native command a maintainer would actually run (e.g. `php artisan migrate`, `mix ecto.migrate`, `diesel migration run`), so the output is actionable rather than abstract. The multi-ecosystem detector is proven on the real `laravel/laravel` repository and a seven-framework unit matrix.

5.2 NoSQL change detection across five engines

Destructive data changes in NoSQL stores look nothing like `DROP TABLE`. PATCHLINE recognizes schema-change and data-migration operations for MongoDB, Cassandra, Elasticsearch, Redis, and DynamoDB (Table 3), each gated by an engine-specific signal and each tagged with a destructive flag. The detector is proven on a real Cassandra repository (`laaksomavrick/twitter-go`) that contains `DROP KEYSPACE` and `DROP TABLE` operations, plus a five-engine unit matrix with an explicit no-false-positive test over prose.

Table 3: NoSQL engines and representative destructive operations flagged by PATCHLINE.

Engine	Signal	Destructive operations
MongoDB	`db.<coll>`, migrate-mongo	drop, dropDatabase, deleteMany, $unset, renameCollection
Cassandra	`.cql`, `KEYSPACE`	DROP KEYSPACE, DROP TABLE, TRUNCATE, ALTER…DROP
Elasticsearch	`_bulk`, `_mapping`	index DELETE, _delete_by_query, bulk delete
Redis	`redis-cli`, `.redis`	FLUSHALL, FLUSHDB, DEL, RENAME
DynamoDB	`dynamodb`	delete-table, batch DeleteRequest

5.3 Data-pipeline repair evidence across four frameworks

Data also moves and reshapes through orchestration and compute frameworks. PATCHLINE records data-pipeline changes for Airflow DAGs (backfills, dagrun resets, embedded destructive SQL), dbt models (full-refresh and `materialized='table'`), Spark jobs (`.mode("overwrite")`, `saveAsTable`), and Kafka consumers (offset resets). The detector is proven on a real multi-framework lakehouse repository (`harrydevforlife/building-lakehouse`) that combines Airflow orchestration, dbt transforms, and Spark overwrite writes, alongside a four-framework unit matrix.

5.4 Schema-registry and protobuf/Avro compatibility

Serialized data outlives the code that wrote it. PATCHLINE inspects protobuf (`.proto`) and Avro (`.avsc`) schemas, flagging proto2 `required` fields (which cannot be removed safely), messages lacking `reserved` declarations (tag-reuse hazard), and Avro record fields without defaults (which break backward/forward compatibility). It is proven on the real `apache/avro` repository, which contains both Avro fields without defaults and proto2 required fields, with a no-false-positive rule ensuring ordinary JSON is never mistaken for an Avro schema.

5.5 Infrastructure/data ordering

A migration that races its deploy is a classic outage. PATCHLINE performs a derived cross-fact analysis that correlates deploy-ordering markers (Helm hooks, Argo sync-waves, `initContainers`, Terraform `depends_on`/waits) with the migration and database jobs on the same manifest, and classifies each data-change job as *sequenced* (ordered by an explicit marker) or *unordered* (able to race the rollout). On the real `helm/charts` repository, PATCHLINE classifies well-known charts such as Kong and Anchore as sequencing their migration jobs with Helm pre/post-upgrade hooks, while other database jobs run unordered.

5.6 Monorepo package boundaries

PATCHLINE detects package boundaries for Bazel, Pants, Nx, Turborepo, Maven, Gradle, and Go workspaces, with a no-false-positive rule that ignores build files outside a workspace. This scopes data-change analysis to the right package in a large monorepo; it is proven on the real `vercel/turbo` example workspace.

Summary. Table 4 consolidates the surfaces, the fact kinds they emit, and the real repository each is proven against.

Table 4: Data-change surfaces, emitted fact kinds, and the real repository each is proven on.

Surface	Fact kinds	Real-repo proof
Relational migrations	`schema_evolution`, `migration_root`	`laravel/laravel`, `lobsters/lobsters`
NoSQL changes	`nosql_change`	`laaksomavrick/twitter-go`
Data pipelines	`data_pipeline_change`	`harrydevforlife/building-lakehouse`
Schema compatibility	`schema_compatibility`	`apache/avro`
Infra/data ordering	`infra_data_ordering`	`helm/charts`
Package boundaries	`package_boundary`	`vercel/turbo`

6 Bounded Interventions and Deterministic Rejection

PATCHLINE can generate interventions—typically guards and tests that make a risky data change safer to review—but it does so under strict constraints that embody principles P4 and P1.

Budgets. Generation is bounded by an explicit budget `files=N,lines=N,tokens=N,changes=N`. `changes` limits how many ranked risks are targeted, `files` limits generated artifacts, `lines` bounds each artifact, and `tokens` caps approximate generated output. This makes generation auditable and prevents unbounded edits.

Quarantine. Generated artifacts are written under `patchline-proposals/` as *untrusted*, non-executable files. They are never applied to the working tree and are skipped from native test execution unless the user explicitly passes `-run-native-tests`, which itself only enables an allowlist of safe checks.

Deterministic rejection. The `compare` stage re-analyzes a proposal and accepts it only if the evidence improves (safety up, completeness up, uncertainty down) without regressions. A generated diff that is *plausible but unsafe* is rejected before it can be trusted, and the repository ships gates that demonstrate exactly this: plausible generated diffs being rejected by deterministic re-analysis, and a per-release regression archive that detects an injected safety drop via a negative control.

Generator-agnostic. A local or hosted generator can be plugged in with `-llm-command '<cmd>'`; PATCHLINE sends the prompt on stdin and stores the output as an untrusted artifact for deterministic compare. `-no-llm` disables generation entirely and rejects any generator command, enabling fully template-only, reproducible analysis—the mode used by every gate in this paper.

The accept/reject decision. Table 5 makes the compare contract precise. Each proposed artifact is scored on the same fact-derived axes as the baseline, and the decision is a pure function of the evidence delta—never of a model's self-reported confidence. An artifact that adds a guard but also removes a provenance link, weakens an invariant, or introduces a new destructive fact is rejected, and the rejecting axis is recorded so the decision is auditable.

Table 5: The deterministic compare contract: an artifact is accepted only if no axis regresses.

Axis	Accept requires	On regression
Safety	no new destructive or breaking facts	reject
Completeness	provenance and coverage preserved or improved	reject
Uncertainty	proof holes not widened	reject
Budget	within `files/lines/tokens/changes`	reject

This is the mechanism behind PATCHLINE's central safety claim: generation cannot make the artifact *worse* without being caught, because the same deterministic analyzer that produced the baseline is the judge of every proposal, and its verdict is reproducible byte-for-byte.

7 Runtime and Observability Evidence

Static risk is necessary but not sufficient; a high-severity finding that never executes in production is different from one implicated in an incident. PATCHLINE scores every finding on two *independent* axes—static risk and observed runtime—and integrates real observability formats to populate the runtime axis:

- **OpenTelemetry (OTLP):** generates valid OTLP traces from a baseline, linking each data-change span back to a finding by id and table for offline replay.
- **Datadog-style timelines:** reconstructs an incident timeline (deploy marker, APM spans, logs, monitor alert) with verified *deploy* < *span* < *log* < *alert* ordering and correlation coverage.
- **Prometheus/Grafana:** emits and re-ingests a Prometheus range export and a Grafana dashboard (SLO burn, error-rate, latency), correlating observed SLO breaches back to high-severity findings.

A *confidence-quadrant* model then separates confirmed incidents (high static risk, high runtime evidence) from unconfirmed static risk (high static, low runtime), and reports a divergence metric. This is the antithesis of an AI assistant's single opaque confidence number: the two axes are independently sourced and independently auditable.

7.1 Performance and scale

PATCHLINE is engineered to keep the inner analysis loop fast enough to run inside CI on every pull request and across a large public corpus in a gate sweep. The analyzer is a single statically linked Go binary with no runtime services. A pinned-slice inventory analysis over the `lobsters db/migrate` directory completes in well under a second on a developer laptop, and a local-path inventory pass—the hot path exercised repeatedly by the delta-debugging fixture minimizer—runs in a few hundred milliseconds, which is what makes oracle-guided minimization practical. Fetches are content-addressed and cached, so repeated and resumed runs avoid redundant downloads entirely; the `-resume` path reuses prior fetch, inventory, baseline, proposal, and compare artifacts, and the `offline` mode validates cached inputs with no network at all. Because every artifact is byte-stable for fixed inputs (P1), incremental and parallel corpus execution can safely cache and shard by repository without risking nondeterministic diffs. The build itself is kept honest by a performance-budget gate that fails if the hot analysis path regresses beyond a documented threshold.

8 Reproducibility and the Gate Methodology

The gate methodology is PATCHLINE's central engineering and scientific contribution. A gate is not a unit test; it is an executable claim about real code.

Anatomy of a gate. Each gate (a) validates a versioned, human-readable example specification whose `claim` field states what is being proven in prose; (b) verifies that the capability's documentation and the README still describe it, so docs cannot silently drift from behavior; (c) downloads a pinned public repository slice and runs the real analyzer with `-no-llm`; (d) asserts machine-checkable invariants on the output (e.g. "at least five destructive changes," "the five-engine unit matrix passes," "the minimized fixture still triggers"); and (e) writes a `gate-summary.json` for downstream aggregation.

Robustness. PATCHLINE ships a parser quality dashboard that unifies, per ecosystem, the emitted fact kinds, the real-repo proof gate, the documented known gaps, and a fuzz-seed corpus of malformed and high-byte inputs that the analyzer must process *without a single crash*. In the current build, all six ecosystems carry a real-repository proof and the full fuzz corpus survives.

Minimal reproductions. For every supported ecosystem, PATCHLINE ships a delta-debugging fixture minimizer that uses the real analyzer as an *oracle* to reduce a seed input to the smallest fixture that still reproduces a destructive finding, and then proves the result is *1-minimal* (removing any remaining line drops the target). A real Cassandra migration from `laaksomavrick/twitter-go` is reduced from 17 lines to a single line that still triggers the destructive `nosql_change` fact (Table 6).

Table 6: Fixture minimization results. Each minimized fixture is 1-minimal under the analyzer oracle.

Ecosystem	Seed lines	Minimized	1-minimal
Cassandra (real repo)	17	1	yes
SQL	7	1	yes
Spark pipeline	6	2	yes
Avro schema	9	2	yes

Drift protection. Beyond behavior, gates protect the *narrative*: claims-to-evidence mapping, a limitations ledger, a camera-ready checklist that blocks release when claims, figures, tables, or docs drift from generated evidence, and a changelog discipline gate that requires every user-visible feature to link to a public proof repository and a reproducing gate.

9 Evaluation Methodology

PATCHLINE treats evaluation as a first-class, reproducible subsystem rather than a one-off table. The following gate-backed studies run on real downloaded baselines.

Blinded false-positive adjudication. A three-rater, blinded adjudication of findings reports Cohen's κ, three-way agreement, and the majority-adjudicated false-positive rate. This provides an inter-rater-reliability story that reviewers expect from an empirical claim.

Seeded false-negative recall. Known public-incident hazard analogues are seeded into a real migration layout; PATCHLINE then measures detection recall and surfaces the false negatives it under-escalates or misses, cross-validated on real repository migrations.

Ablations and effect sizes. An ablation study removes provenance links, cross-file context, generated guards, runtime traces, and risk budgets one at a time on a real baseline and reports each signal's measured contribution to the ranking. A companion study reports standardized Cohen's d effect sizes for risk density and hazard-class severity across ecosystem, repository size, and migration framework strata.

Severity calibration. Severity is validated against *independent* danger evidence— incident/cause clusters, rollback/fix migrations, and recurrences—and the calibration lift is reported across real repositories. This guards against a severity score that is internally consistent but externally meaningless.

Stratification. Repositories and migrations are stratified by ecosystem (balanced across nine migrators), repository size (small apps, medium services, monorepos, infrastructure-heavy), and migration age and change type (recent vs. old, schema-only vs. backfill-heavy), with a representative real download proven from each stratum. Longitudinal reruns over multiple historical commits per repository summarize risk/evidence deltas over time.

Baselines. PATCHLINE's value over weaker baselines is made concrete through side-by-side examples in which PATCHLINE links cross-file repair clues that grep-only and SQL-only baselines miss. Table 7 contrasts the three approaches on the capabilities that matter for safe data-change review.

Table 7: Capability comparison against common lightweight baselines.

Capability	**grep-only**	**SQL-linter**	**Patchline**
Destructive DDL detection	partial	yes	yes
NoSQL / pipeline / schema surfaces	no	no	yes
Cross-file provenance links	no	no	yes
Derived infra/data ordering	no	no	yes
Blast-radius & proof-hole ranking	no	no	yes
Runtime-trace corroboration	no	no	yes
Bounded, quarantined interventions	no	no	yes
Deterministic, byte-stable artifacts	no	partial	yes
Real-repo proof gates	no	no	yes

Headline measured results. Table 8 consolidates the principal quantitative results that recur throughout the paper, each regenerated by a gate against a pinned public slice.

Table 8: Headline results, each reproduced by a `make` gate target.

Result	**Evidence**
158 files / 328 ranked risks / 100 provenance slices	`lobsters/lobsters db/migrate`
8 review artifacts, 0 failed deterministic checks	same run, bounded budget
9 Cassandra changes incl. destructive drops	`laaksomavrick/twitter-go`
17 $\rightarrow$ 1 line, 1-minimal reproduction	delta-debugging oracle
3 pipeline frameworks in one repo	`harrydevforlife/building-lakehouse`
10 sequenced / 3 unordered data jobs	`helm/charts`
dozens of schema-evolution hazards	`apache/avro`
6/6 ecosystems fuzz-clean (0 crashes)	parser dashboard corpus
159 gates / 116 docs / 92 examples	repository inventory

10 Developer and Contributor Experience

A research artifact only earns stars if it is pleasant to extend. PATCHLINE ships a scaffolder (`scripts/new-ecosystem.sh`) that generates the example specification, worker script, gate script,

and documentation stub for a new ecosystem following the established pattern, and an *onboarding quest*—a six-step path from scaffold to a passing real-repo gate that a contributor can complete in under one hour. A dedicated gate proves the scaffolder emits valid, runnable artifacts and that the quest narrative stays in sync with the required steps. PATCHLINE also exposes deterministic plugin interfaces (parser, fact extractor, linker, ranker, proposal generator, compare check, report renderer) and a `golden-fixture` command that turns a real-repo slice into a tiny deterministic Go test without vendoring the full repository.

11 Case Studies

`lobsters/lobsters` (Rails, `db/migrate`). On a pinned commit, PATCHLINE scans 158 files and surfaces 328 ranked data-change risks with 100 provenance slices, generating 8 review artifacts with zero deterministic checks failed. This is the headline landing demo and is itself gate-protected so the README's numbers cannot drift.

`apache/avro` (schema compatibility). PATCHLINE flags dozens of Avro record fields without defaults and multiple proto2 required fields as schema-evolution hazards, demonstrating that the same risk lens applies to serialization schemas, not just databases.

`helm/charts` (infra/data ordering). PATCHLINE classifies ten migration/database jobs as sequenced by explicit Helm hooks (including Kong and Anchore migrations) and three as unordered, showing that ordering risk is recoverable from real infrastructure manifests.

`harrydevforlife/building-lakehouse` (pipelines). A single repository exercises three pipeline frameworks at once—Airflow backfills, dbt table materializations, and Spark overwrite writes—and PATCHLINE records the destructive operations across all three as uniform pipeline facts.

`laaksomavrick/twitter-go` (NoSQL + minimization). PATCHLINE detects nine Cassandra changes including destructive `DROP KEYSPACE`/`DROP TABLE`, then delta-debugs a real migration to a 1-minimal reproduction.

12 Related Work

PATCHLINE sits at the intersection of several tool families but is distinguished from each. *Migration linters* (e.g. online-schema-change advisors, framework-specific guards) typically check a single statement or a single ecosystem; PATCHLINE unifies nine relational ecosystems with NoSQL, pipelines, schemas, and infrastructure under one fact model. *Code scanners* target injection, secrets, and CWE classes in application code; PATCHLINE targets the orthogonal axis of destructive data change and carries provenance for each finding. *Observability platforms* show what happened at runtime but do not statically rank data-change risk; PATCHLINE integrates their export formats to populate an independent runtime-evidence axis. *AI coding assistants* generate migrations and guards but cannot prove safety; PATCHLINE treats generation as bounded, quarantined, and subject to deterministic rejection. Finally, PATCHLINE's *gate methodology*—executable claims against pinned public code with drift protection—is a reproducibility contribution in its own right. Table 9 summarizes the positioning.

Table 9: Positioning of PATCHLINE relative to neighboring tool families.

Property	Migr. linter	Code scanner	Observability	Patchline
Polyglot data surfaces	no	no	no	yes
Provenance per finding	partial	partial	yes	yes
Static risk ranking	partial	yes	no	yes
Runtime corroboration	no	no	yes	yes
Safe generation	no	no	no	yes
Reproducible proof gates	no	no	no	yes

13 Threats to Validity

Construct validity. PATCHLINE operationalizes "data-change risk" as a ranked set of facts with explicit destructiveness and blast-radius properties rather than as a single ground-truth label. To keep this construct honest, severity is calibrated against *independent* danger evidence (incident clusters, rollback/fix migrations, recurrences) rather than against PATCHLINE's own score, and the calibration lift is reported. The blinded three-rater false-positive adjudication, with Cohen's κ and three-way agreement, further guards against a definition of "risk" that only PATCHLINE agrees with.

Internal validity. Because every artifact is byte-stable for fixed inputs and every claim is regenerated by a gate against a pinned commit, the results reported here are not subject to run-to-run variance or silent environmental drift; a regression in code, documentation, or an upstream ref fails the corresponding gate loudly. The ablation study isolates each signal's contribution by removing provenance links, cross-file context, generated guards, runtime traces, and risk budgets one at a time, so the reported value of each component is measured rather than asserted.

External validity. The evaluation deliberately spans nine relational ecosystems plus NoSQL, pipeline, schema, and infrastructure surfaces, and stratifies by ecosystem, repository size, and migration age/change type with a real download proven from each stratum. This breadth mitigates, but does not eliminate, the risk that results are specific to the chosen public repositories; longitudinal reruns over multiple historical commits per repository test temporal stability.

Known blind spots. The seeded false-negative study explicitly measures what PATCHLINE misses by injecting known public-incident hazard analogues into real layouts and reporting recall, and the limitations ledger (Section 14) publishes the documented heuristic boundaries rather than hiding them.

14 Limitations

PATCHLINE is deliberately conservative and therefore has documented blind spots, which it publishes in a limitations ledger rather than hiding. Dynamic SQL assembled at runtime is only partially recovered; vendor-specific NoSQL aggregation pipelines are matched heuristically rather than fully parsed; dynamically generated DAGs and templated SQL are detected only when literals

are present; cross-file Helm/Argo dependencies are not yet linked into a single ordering graph; and per-message protobuf field-number uniqueness is not yet diffed across versions. Source provenance for very large monorepos fetched over Mercurial/Fossil without a native binary is approximated by content hashing. PATCHLINE ranks and explains risk; it does not certify a migration as safe, and it intentionally avoids full-repair claims.

15 Discussion

Why a fact layer instead of a model. A recurring question is why PATCHLINE does not simply fine-tune a model to classify risky diffs. The answer is auditability. A reviewer approving a destructive migration must be able to see *why* a change was flagged, trace it to a specific line, and reproduce the judgment tomorrow. A fact—kind, path, rationale, typed identifiers, destructiveness flag—is a unit of explanation, not just a unit of classification. The uniform envelope also means that adding a new surface (say, a sixth NoSQL engine) extends coverage without changing how ranking, diffing, or reporting work, which is what keeps the system tractable as breadth grows.

Why gates instead of unit tests. Unit tests pin behavior against fixtures the author chose; gates pin behavior against *real, pinned public repositories* the author did not control, and additionally assert that the documentation and README still describe the behavior. This catches a class of failures unit tests cannot: silent capability drift, documentation that overstates the tool, and upstream changes that invalidate a claim. The cost is that gates are heavier and require network access to a pinned slice; PATCHLINE mitigates this with content-addressed caching and an offline mode so that gate sweeps are fast and repeatable.

Generalization beyond data changes. While PATCHLINE targets data-change risk, the architecture—deterministic stages, a provenance-carrying fact layer, bounded quarantined generation, and gate-backed claims—is domain-agnostic. The plugin interfaces (parser, fact extractor, linker, ranker, proposal generator, compare check, report renderer) make it plausible to retarget the same machinery at other high-stakes, weak-oracle domains such as access-control changes or feature-flag rollouts. We leave such retargeting to future work but note that nothing in the core couples it to databases specifically.

On the cost of conservatism. Principle P3 (conservative by construction) means PATCHLINE accepts false negatives to avoid confident false positives. This is the right trade for a tool whose output gates a human reviewer's attention: a noisy analyzer is quickly ignored, whereas a quiet, high-precision analyzer that occasionally misses a hazard still raises the floor on review quality. The seeded false-negative study exists precisely to keep this trade visible and measurable rather than implicit.

16 Future Work

Several extensions follow naturally from PATCHLINE's architecture. First, the cross-file infrastructure ordering analysis currently classifies each data job independently; linking Helm hooks, Argo sync-waves, and `initContainers` into a single repository-wide ordering graph would let PATCHLINE detect ordering hazards that span multiple manifests rather than only per-job markers. Second, schema compatibility is presently evaluated within a single revision; diffing protobuf and Avro schemas *across* versions—tracking field-number reuse, type narrowing, and default changes over a

commit range—would turn the static schema lens into a true compatibility checker. Third, the runtime axis ingests exported telemetry; a streaming mode that correlates live traces against the static baseline in near-real-time would tighten the confirmed-incident quadrant. Fourth, the plugin interfaces invite retargeting the deterministic-fact-plus-gate methodology at adjacent weak-oracle domains such as access-control and feature-flag changes. Finally, the evaluation harness is positioned to grow into a public, longitudinal leaderboard that reruns the full gate suite across an expanding corpus of pinned public repositories, turning reproducibility from a property of this paper into an ongoing community benchmark.

17 Conclusion

PATCHLINE demonstrates that the highest-risk changes teams ship—destructive, polyglot, cross-cutting data changes—can be analyzed *deterministically*, with full provenance, across relational, NoSQL, pipeline, schema, and infrastructure surfaces, and that generation can be made safe by bounding it, quarantining it, and deterministically rejecting it. Just as important, PATCHLINE shows that *every* such claim can be made reproducible: 159 gates prove the system against real, pinned public code and fail loudly on drift, and a research-grade evaluation methodology—blinded adjudication, seeded recall, ablations, effect sizes, and severity calibration—turns the artifact into evidence. We believe this combination of breadth, safety, and gate-backed reproducibility is a template for trustworthy program-analysis artifacts.

Availability. PATCHLINE is a single Go binary with no required external services. Every result in this paper is regenerated by a `make` target against pinned public repositories.

A A Worked End-to-End Example

To make the pipeline concrete, we walk through a single invocation against a pinned slice of the `lobsters/lobsters` Rails application. The command requests the full pipeline with a bounded budget and template-only generation:

```
go run ./cmd/patchline repo analyze \
  -github lobsters/lobsters \
  -ref 3b80b47aa5aaba37ec44413e7d1dc96fcf1585b6 \
  -subpath db/migrate \
  -stages inventory,baseline,propose,compare \
  -proposal-kind all \
  -budget files=8,lines=100,tokens=12000,changes=2 \
  -no-llm -out results/generated/landing-demo -json
```

Fetch. PATCHLINE resolves the pinned commit, downloads the `db/migrate` slice, and writes `source.json` recording the GitHub host, the resolved 40-character commit, and the archive SHA-256. Because the commit is pinned, every subsequent run is reproducible.

Inventory. PATCHLINE scans 158 files, identifies the Rails migration system and its native command, and emits the fact stream. Schema-evolution facts are derived directly from the migration bodies—no pre-authored schema is required—so an `ALTER TABLE ... DROP COLUMN` becomes a `schema_evolution` fact carrying the table and column as typed identifiers.

Baseline. PATCHLINE ranks 328 data-change risks, attaches 100 provenance slices, estimates blast radius, and minimizes proof holes. The output is written both as `baseline.json` and as `baseline.sarif` for IDE and CI ingestion.

Propose and compare. Within the budget, PATCHLINE generates 8 review artifacts (guards and tests) under `patchline-proposals/`, all non-executable. The `compare` stage re-analyzes them; in this run zero deterministic checks fail, and the accept/reject decision for each artifact is recorded with its evidence delta. The headline numbers—158 files, 328 risks, 100 provenance slices, 8 artifacts, 0 failed checks—are themselves asserted by the `landing-readme-gate` so the README cannot drift from reality.

B Fact Schema Reference

Every fact shares the schema in Table 10. Detectors differ only in the `kind` and the `properties` they populate; the uniform envelope is what allows ranking, diffing, and tabulation to be detector-agnostic.

Field	Meaning
`version`	Fact schema version, enabling forward-compatible evolution.
`id`	Stable content-derived identifier for cross-referencing (e.g. from a trace span).
`kind`	The detector category, e.g. `schema_evolution`, `nosql_change`, `data_pipeline_change`, `schema_compatibility`, `infra_data_ordering`.
`path`	Repository-relative path of the evidence.
`confidence`	`observed` for directly matched signals, `derived` for cross-fact inferences.
`rationale`	Human-readable justification; the basis of PATCHLINE's explainability.
`identifiers`	Typed identifiers (table, model, engine, operation, job, ...) for linking.
`properties`	Detector-specific key/values, including the `destructive` or `breaking` flag.

Table 10: The uniform fact envelope shared by all detectors.

C Detector Rule Catalog

Tables 11–12 list representative rules for the pipeline and schema-compatibility detectors. Each rule is signal-gated (P3): the framework or format signal must match before any rule is evaluated, which is what keeps prose and unrelated code from being misclassified.

D Infrastructure/Data Ordering Markers

The ordering analysis (Section 5) treats the markers in Table 13 as evidence that a data-change job is sequenced relative to its rollout. A migration or database job co-located with any such marker is classified *sequenced*; one with none is *unordered*.

Table 11: Data-pipeline detector: signals and representative operations.

Framework	Signal	Destructive operations
Airflow	`from airflow`, `DAG(`	embedded DROP/TRUNCATE, backfill, `clear_task_instances`, dagrun reset
dbt	`{{ config(`, `{{ ref(`	`-full-refresh`, `materialized='table'`
Spark	`pyspark`, `SparkSession`	`.mode("overwrite")`, `saveAsTable`, `insertInto`
Kafka	`KafkaConsumer`, `@KafkaListener`	`auto.offset.reset`, `seekToBeginning`, `-reset-offsets`

Table 12: Schema-compatibility detector: formats and breaking risks.

Format	Inspected	Breaking risk
protobuf	message fields, syntax level	proto2 `required` field; message with no `reserved`
Avro	record fields	field without a `default`
registry	`buf.yaml`, `buf.gen.yaml`	compatibility-policy presence

E Selected Gate Catalog

PATCHLINE ships 159 gates; Table 14 lists a representative selection grouped by theme to convey the breadth of what is proven against real, pinned public code. Each gate is a single `make` target.

Theme	Representative gates
Source breadth	`mercurial-fossil-source-gate`, `monorepo-boundary-gate`
Migration breadth	`multi-ecosystem-migration-gate`, `nosql-change-gate`, `data-pipeline-gate`, `schema-compat-gate`, `infra-ordering-gate`
Intervention safety	`generated-code-quarantine-gate`, `rejected-generated-gate`, `intervention-regression-archive-gate`, `reviewability-examples-gate`
Runtime evidence	`otel-trace-gen-gate`, `datadog-timeline-gate`, `prom-grafana-gate`, `runtime-confidence-gate`
Evaluation rigor	`fp-adjudication-gate`, `fn-discovery-gate`, `ablation-study-gate`, `effect-size-strata-gate`, `severity-calibration-gate`
Stratification	`ecosystem-balanced-benchmark-gate`, `repository-size-stratification-gate`, `migration-age-stratification-gate`, `longitudinal-public-reruns-gate`
Security & privacy	`secret-scan-gate`, `archive-security-gate`, `threat-model-gate`, `redaction-stability-gate`, `privacy-metrics-gate`, `supply-chain-provenance-gate`

Theme	Representative gates
Robustness & DX	`fuzz-coverage-gate`, `parser-dashboard-gate`, `fixture-minimizer-gate`, `onboarding-quest-gate`, `performance-budget-gate`
Artifact & paper	`reviewer-walkthrough-gate`, `artifact-evaluation-kit-gate`, `paper-figures-gate`, `paper-appendix-gate`, `camera-ready-checklist-gate`

Table 14: A representative selection from the 159-gate catalog, grouped by theme.

F Security and Privacy Model

Because PATCHLINE ingests *untrusted* repositories and archives, it carries an explicit threat model that is itself gate-verified. Archive extraction defends against path traversal, symlink escape, malformed archives, and archive bombs, while still extracting pinned public GitHub archives through a content-addressed cache. Generated proposal files are non-executable, unapplied, and excluded from native execution unless the user explicitly enables an allowlist of safe checks. Redaction is byte-stable across repeated and resumed runs, secret-scanning proves that redacted reports, prompts, bundles, and CI artifacts do not leak deterministic canary values, and an aggregate privacy-metrics mode emits source-free risk trends with salted cohort identifiers so results can be shared without raw evidence. Binaries, release archives, and public-corpus downloads carry deterministic provenance and sorted checksums.

G Reproduction Guide

Every claim in this paper is regenerated by a `make` target. To reproduce the breadth results of Section 5 against pinned public repositories:

```
make multi-ecosystem-migration-gate   # Laravel + 7-framework matrix
make nosql-change-gate       # Cassandra DROP on twitter-go + 5-engine matrix
make data-pipeline-gate      # Airflow+dbt+Spark on building-lakehouse
make schema-compat-gate      # Avro + protobuf on apache/avro
make infra-ordering-gate     # Kong/Anchore sequencing on helm/charts
make fixture-minimizer-gate  # 1-minimal reproductions per ecosystem
make parser-dashboard-gate   # coverage + fuzz robustness dashboard
```

Each target downloads only the pinned slice it needs, runs the real analyzer with `-no-llm`, asserts its invariants, and writes a `gate-summary.json`. A failing assertion—whether from a code regression, a documentation drift, or a changed upstream ref—fails the target loudly, which is precisely the property that makes the artifact trustworthy.

Table 13: Deploy-ordering markers correlated with migration/database jobs.

Marker	**Source**
`helm.sh/hook` (pre/post-install, pre/post-upgrade)	Helm
`argocd.argoproj.io/sync-wave`	Argo CD
`initContainers`, `depends-on`	Kubernetes
`depends_on`, `wait = true`, `atomic = true`	Terraform / Helm provider